



Neural network for ordinal classification of imbalanced data by minimizing a Bayesian cost

Marcelino Lázaro*, Aníbal R. Figueiras-Vidal

Signal Theory and Communications Dept. Universidad Carlos III de Madrid, Spain

ARTICLE INFO

Article history:

Received 2 February 2022

Revised 1 December 2022

Accepted 4 January 2023

Available online 6 January 2023

Keywords:

Bayes cost

Parzen windows

Ordinal classification

Imbalanced

ABSTRACT

Ordinal classification of imbalanced data is a challenging problem that appears in many real world applications. The challenge is to simultaneously consider the order of the classes and the class imbalance, which can notably improve the performance metrics. The Bayesian formulation allows to deal with these two characteristics jointly: It takes into account the prior probability of each class and the decision costs, which can be used to include the imbalance and the ordinal information, respectively. We propose to use the Bayesian formulation to train neural networks, which have shown excellent results in many classification tasks. A loss function is proposed to train networks with a single neuron in the output layer and a threshold based decision rule. The loss is an estimate of the Bayesian classification cost, based on the Parzen windows estimator, which is fitted for a thresholded decision. Experiments with several real datasets show that the proposed method provides competitive results in different scenarios, due to its high flexibility to specify the relative importance of the errors in the classification of patterns of different classes, considering the order and independently of the probability of each class.

© 2023 The Author(s). Published by Elsevier Ltd.

This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

1. Introduction

Ordinal classification is a special form of multi-class classification where the classes exhibit an inherent ordering. It is a frequent problem because a natural ordering occurs in many human tasks, such as those associated with every kind of grading in human devised scales: satisfaction surveys, medical diagnosis, quality assessment or credit rating are just some examples (a detailed list of applications in different research areas can be found in the literature review [1]). Because of its relevance, although this problem has been studied for more than four decades [2] ordinal classification is still receiving attention in several different directions: ordinal classification methods (see the taxonomy in [1]), appropriate performance metrics [3,4] or the interpretability of neural classifiers for ordinal problems [5,6] are some examples.

There is a wide variety of ordinal classification methods: from initial approaches that stated the problem as a conventional regression problem [7], where the ordinal classes were mapped into sorted numeric values, to more recent procedures that update con-

ventional techniques to include the ordinal constraints (such as Gaussian Processes (GP) [8], Support Vector Machines (SVM) [9], Neural Networks (NN) [10], or Learning Vector Quantizers (LVQ) [11]). In the survey [1] these methods are grouped into three categories:

- *Naive approaches*: The problem is posed as another standard problem, such as regression or nominal classification, without considering the ordering of the classes (with evident limitations), or as a cost-sensitive classification problem, with different costs for different misclassification errors. How to determine the cost matrix without a priori knowledge of the problem is the main limitation of these techniques.
- *Ordinal binary decomposition approaches*: The ordinal target variable is mapped into several binary variables, which are used to train a single multiple-output model or several models. In many cases, the binary decomposition violates the monotonicity or consistency of the ordinal target, although consistent solutions have also been proposed [12].
- *Threshold methods*: They are based on the definition of thresholds over a real valued measure (or latent variable) that will define the intervals associated to each class. To force a monotonic behavior in this latent variable according to the ordinal structure of the problem can be helpful to improve the generalization ability of the trained model. A potential problem is

* Corresponding author at: Departamento de Teoría de la Señal y Comunicaciones, Universidad Carlos III de Madrid. Av. Universidad 30, 28911 Leganés, Madrid, Spain.

E-mail address: mlazaro@tsc.uc3m.es (M. Lázaro).

that the distances among the ordered classes are not known a priori. Because of this, most methods estimate these distances during the learning process.

Many real ordinal problems present a strong imbalance among classes. Conventional classifiers tend to be biased towards majority classes and to produce more errors for the minority classes, although minority events are often highly relevant in many applications (medical diagnosis, fraud detection, etc.). Imbalanced problems are frequent and relevant, and consequently several methods have been proposed to solve them. The proposed solutions can be grouped into three categories [13]: *Data preprocessing methods*, such as resampling or data augmentation techniques (SMOTE [14] is probably the best known technique). The effectiveness of sampling techniques depend on the problem and on the nature of the classifier, they can modify the class distributions, and in general there is not guarantee of an improved performance. *Cost-sensitive methods* modify the decision thresholds or assign weights to instances according to a cost matrix, or are trained with an objective function that is cost sensitive, as active learning versions [15]. The Bayes decision theory has also been used in this context, usually with a learning method used to estimate the class posterior probabilities (although most of these estimates are not principled, which spoils the robustness of these approaches). Ensemble methods, which combine multiple classifiers, are also used to deal with imbalanced problems [16,17], including binarization techniques [18]. Most of these procedures are empirical and, consequently, they implicate degradation risks.

Although imbalance appears in many real ordinal problems, few works have proposed solutions considering the ordinal and the imbalance characteristics simultaneously. In most cases, methods are focused on ordinal classification and imbalance is addressed using sampling techniques [19,20] or ensembles [21]. Also very few works, such as [21], have evaluated the performance with appropriate figures of merit for this scenario.

Our objective is to propose a principled solution for ordinal and imbalanced problems, and to evaluate it using appropriate metrics. The proposed solution is based on the combination of neural networks with the Bayes decision theory. Neural networks have shown excellent performance in many applications, including ordinal [10] and imbalanced problems [22] (further details of the use of NNs for ordinal and imbalanced problems can be found in Section 2). The Bayes classification cost considers the class probabilities and the decision costs, which allow to introduce the imbalance and the ordinal information, respectively, in a solid statistical framework. We propose to combine these two powerful tools to create an ordinal classification method capable of dealing with imbalanced data directly, without using data preprocessing techniques or ensembles. The proposed method can be applied to any neural network architecture with a one-dimensional output and a threshold-based decision (the thresholds will be learned, to deal with the lack of knowledge about the relative distance among classes). Our main contribution is a new loss function that is an estimate of the Bayes classification cost, using the Parzen windows estimator to model the conditional probability of the network output for each class. A similar approach was proposed in [23] for imbalanced binary classification, and later extended in [24] for binary example-dependent-cost (EDC) classification. Here, the formulation is fitted to a multiclass ordinal problem with a thresholded decision rule.

The manuscript is organized as follows. Section 2 reviews some related works. Section 3 formally states an imbalanced ordinal problem. The proposed solution is presented in Section 4. Experiments with real datasets are presented in Section 5 to evaluate the proposed method, and Section 6 closes the manuscript summarizing its characteristics.

2. Related works

There are many works dedicated to ordinal classification problems with imbalanced data, but few works consider the ordinal information and the imbalance jointly. In this section, related works are reviewed to highlight the differences with the proposed solution. Because of this, the review is focused on neural network solutions, cost-sensitive approaches and loss functions used in ordinal classification.

2.1. Preprocessing methods or ensembles to handle imbalance

Many solutions are focused on ordinal classification, and the imbalance is handled using preprocessing techniques, such as sampling or data augmentation, or ensembles. Some examples, [19–21], were mentioned above, and additional examples will be addressed in the following sections.

In our work the imbalance is considered in the loss function to be minimized in the training of a neural network, which also considers the order information.

2.2. Use of ordinal evaluation metrics

Some methods propose to use ordinal evaluation metrics as a validation tool, but the ordinal information is not included in the design of the classifier, such as [25] where ensembles (stacked generalization) were validated using several ordinal metrics. The imbalance was treated by using sampling techniques.

In this work the ordinal information, as well as the imbalance, is considered in the design of the solution. Appropriate evaluation metrics for an imbalanced ordinal classification problem will be used to evaluate the proposed method.

2.3. Use of the Bayes theory

The Bayes classification theory has been used in the context of machine learning. Some cost-sensitive approaches use the output of a learning machine to provide class probability estimates, and a risk function based on these estimates along with a cost matrix is used to make decisions, as in [26]. However, not all learning machines are able to provide consistent estimates of posterior probabilities [27], and in practice some heuristic calibration procedures are used to improve the performance, as in [28], which harms the robustness of these approaches.

There are also previous approaches using the Parzen windows estimator with the Bayes theory, as in the probabilistic neural networks proposed by Specht [29], or in [30]. The proposed approach is notably different: Parzen windows are applied here at the one-dimensional output space of a neural network instead of being applied in the multi-dimensional input space, as in [29] or [30]. Moreover, in these works the Parzen estimator is used in the decision rule, which is based on its estimates. In the proposed approach, the decision is based on thresholds defining decision regions over the one-dimensional output of a neural network. The Parzen window estimator is used in the definition of the loss function used to train the neural network classifier.

2.4. Ordinal loss functions

Common loss functions used to train neural networks, such as cross-entropy, do not consider the relative ordering among classes or the imbalance (although weighted versions of these losses have been proposed to deal with imbalance). For this reason, several types of loss functions have been proposed for ordinal classification.

Some works, as [31], use losses modeling the output probabilities as following a unimodal distribution. In [32] these losses, along with additional quasi-unimodal losses, are evaluated using a convolutional neural network (CNN) in 3 ordinal datasets of RGB images using several metrics. In this work, the imbalance is only considered in the evaluation phase, by using appropriate evaluation metrics.

In [33], an ordinal loss function was proposed to train CNNs. This loss function adds the cross-entropy per output in a multi-output problem defined by decomposing the M -class ordinal problem in $M - 1$ binary problems. Authors implemented several configurations, including ensembles to evaluate the performance in imbalanced datasets. Similarly, [12] combines a binary decomposition with a weighted cross-entropy loss for $M-1$ binary classifiers with theoretical guarantees for classifier consistency to train CNNs. The weights in the loss for the $M-1$ tasks can be used to take into account the imbalance, with the limitations given by the binary decomposition. However, all experiments were performed with uniform weights and the evaluation metrics, mean absolute error (MAE) and root mean squared error (RMSE), did not consider imbalance.

The quadratic weighted kappa (QWK) metric has been proposed as a loss function to train CNNs [34]. The kappa index, originally designed as a measure of agreement between observations that allows to discard agreements due to mere chance, has been used to evaluate the performance in ordinal problems because it takes into account the difference between the decision and the correct class. In [34] the imbalance is attacked by using data augmentation techniques. In [35] the QWK loss was combined with cumulative link models (CLM) to significantly improve the ordinal classification metrics. Again, augmentation techniques were used to deal with imbalance.

Our proposed loss function is an estimate of the Bayes classification cost, which allows to include in the training of a neural network the ordinal information along with the imbalance, providing also flexibility to consider the relative importance of every class in the problem at hand. It can be used with any neural network architecture, shallow or deep, having a single neuron in the output layer and a threshold-based decision rule.

3. Ordinal classification of imbalanced data

In this section the problem of ordinal classification of imbalanced data is stated. Standard notation will be used: $p_X(x)$ denotes the point mass function of a discrete random variable X , i.e., $p_X(x) = \Pr(X = x)$, $f_X(x)$ is the probability density function (PDF) of a continuous random variable X , $|\cdot|$ is the absolute value operator when applied to scalars or the cardinality of a set, and $\hat{a}$ denotes the estimate or the decision that is made for a .

A multiclass classification problem can be stated as follows: the objective is to assign a given pattern of dimension A , $\mathbf{x} \in \mathbb{R}^A$, to one class out of a set of M possible classes or hypotheses

$$\mathcal{H} = \{H_1, H_2, \dots, H_M\}. \quad (1)$$

In an ordinal problem the classes exhibit a natural order, which is denoted as

$$H_1 < H_2 < \dots < H_M, \quad (2)$$

where $<$ expresses a relation of order. To simplify the notation, in the following $H_i = i$ will be assumed. Note that in this case the cumulative distribution function (CDF) of the classes is well defined and naturally contains the order.

The available information about the problem is a set of labeled samples (typically named the training set)

$$\mathcal{T} = \{(\mathbf{x}_k, y_k) \mid k \in \mathbb{Z}, 1 \leq k \leq N\}, \quad (3)$$

with $y_k \in \mathcal{H}$ being the label that indicates the class the pattern $\mathbf{x}_k$ belongs to. The set $\mathcal{S}_t$ is the set of indexes of patterns that belong to class t

$$\mathcal{S}_t = \{k \mid y_k = t, y_k \in \mathcal{T}\}, t \in \mathcal{H}, \quad (4)$$

and N_t is the number of elements in the set $\mathcal{S}_t$, i.e., $N_t = |\mathcal{S}_t|$. Obviously,

$$\sum_{t=1}^M N_t = N. \quad (5)$$

If the problem is imbalanced, the numbers of samples of the M classes, N_t , $t \in \mathcal{H}$, are significantly different.

4. Proposed method

In this section we will present the proposed solution to ordinal classification of imbalanced data.

4.1. Neural network architecture and decision rule

In conventional classification problems, when a neural network is trained with a labeled data set, multiclass problems are solved typically by using networks with a neuron per class in the output layer with softmax activation (the decision for a pattern is the class associated to the neuron with the highest output for this pattern). To solve the ordinal classification problem we propose to use a neural network with a single neuron in the output layer (one-dimensional output space). To have a single output with a monotonic decision rule (the higher the output, the higher the class label in the ordinal arrangement) allows to match the network output with the ordinal structure of the corresponding problem. For a given input pattern $\mathbf{x}_k$ the output of the network is

$$z_k = g(\mathbf{x}_k, \mathbf{w}) \in \mathbb{R}, \quad (6)$$

where the function $g(\cdot, \mathbf{w})$ depends on the neural network architecture through the set of parameters of the network, that are included in the vector $\mathbf{w}$. From this one-dimensional network output, the classifier has to decide a class for pattern $\mathbf{x}_k$

$$\hat{y}_k = \text{decision}(z_k). \quad (7)$$

In the proposed scheme the decision is based on thresholds defining decision regions. In particular, $M - 1$ ordered thresholds, $\{u_1, u_2, \dots, u_{M-1}\}$, with $u_{d-1} < u_d$, define M decision regions, as shown in Fig. 1

$$I_d = (u_{d-1}, u_d], d \in \mathcal{H}. \quad (8)$$

With these decision regions, the decision rule of the proposed method is

$$\hat{y}_k = d \text{ if } z_k \in I_d. \quad (9)$$

The $M - 1$ thresholds are included in the vector $\mathbf{u}$. To have an appropriate definition for regions I_1 and I_M in (8), we can define $u_0 = -\infty$ and $u_M = +\infty$.

4.2. Loss function: An estimate of the Bayes cost

To simultaneously deal with imbalance and ordinal classification a new loss function, inspired by the Bayes decision theory, is

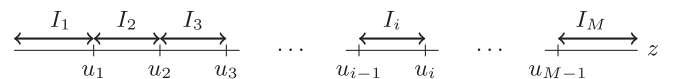


Fig. 1. Thresholds and decision regions for the decision rule (9).

proposed: An estimate of the Bayes cost is fitted to the NN architecture and decision rule presented in the previous section. The Bayesian classification theory is based on the average Bayes cost

$$\mathcal{B}_C = \sum_{t \in \mathcal{H}} \pi_t \sum_{d \in \mathcal{H}} c_{d,t} p_{\hat{Y}|Y}(d|t), \quad (10)$$

which has to be minimized by the classifier. The variable Y represents the true class, and $\hat{Y}$ represents the decision. This formulation is convenient for imbalanced ordinal classification problems: it permits to include the prior probabilities of each class, $\pi_t \equiv p_Y(t)$, and the costs of deciding class d when the true class is t , $c_{d,t}$ (the conditional probability distribution $p_{\hat{Y}|Y}(d|t)$ characterizes the probability of this situation). In this way, it can deal with the imbalance and the relative importance of each class and, at the same time, it allows to specify the impact of the ordinal structure in the classification errors. For instance, making costs proportional to the difference between the decision and the true class, $c_{d,t} \propto |d - t|$, can help to force this ordinal structure in the classifier. For these reasons, in this work we propose a new loss function to train neural networks that is based on the Bayes cost in (10).

In the proposed neural network classifier, with decision rule (9), the conditional probabilities of deciding class d when the true class is t are obtained by integrating the conditional distribution of the output of the network for samples of the true class, $f_{Z|Y}(z|t)$, in the corresponding decision region

$$p_{\hat{Y}|Y}(d|t) = \int_{I_d} f_{Z|Y}(z|t) dz. \quad (11)$$

However, in practice the distributions $f_{Z|Y}(z|t)$ are unknown. We propose to use instead the estimate provided by the Parzen windows method [36]

$$\hat{f}_{Z|Y}(z|t) = \frac{1}{N_t} \sum_{k \in S_t} k(z - z_k), \quad (12)$$

where the function $k(z)$ is the Parzen window, which has to be a valid PDF (nonnegative and with unit area). The most common window is a Gaussian PDF with variance σ^2

$$k(z) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{z^2}{2\sigma^2}}, \quad (13)$$

but any other valid PDF can be used. The Parzen windows method has been chosen because it is a non-parametric estimator that allows to obtain a closed form expression for the gradient of the estimated Bayesian cost, as it will be shown later.

By defining the integral of the Parzen window

$$K(z) = \int_{-\infty}^z k(x) dx, \quad (14)$$

taking into account that

$$\int_{-\infty}^{u_i} k(z - z_k) dz = K(u_i - z_k) \quad (15)$$

and introducing the Parzen windows estimator (12) in (11), the estimate of $p_{\hat{Y}|Y}(d|t)$ becomes

$$\begin{aligned} \hat{p}_{\hat{Y}|Y}(d|t) &= \frac{1}{N_t} \int_{I_d} \sum_{k \in S_t} k(z - z_k) dz \\ &= \frac{1}{N_t} \sum_{k \in S_t} [K(u_d - z_k) - K(u_{d-1} - z_k)]. \end{aligned} \quad (16)$$

This results follows from¹

¹ Note that $\int_{I_1} k(z - z_k) dz = K(u_1 - z_k)$ and $\int_{I_M} k(z - z_k) dz = 1 - K(u_{M-1} - z_k)$, given that $K(-\infty) = 0$ and $K(\infty) = 1$ by definition.

$$\int_{I_d} k(z - z_k) dz = \int_{u_{d-1}}^{u_d} k(z - z_k) dz = K(u_d - z_k) - K(u_{d-1} - z_k). \quad (17)$$

Finally, by introducing (16) in (10) the estimate of the Bayes cost is

$$\hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u}) = \sum_{t=1}^M \frac{\pi_t}{N_t} \sum_{k \in S_t} \left(c_{M,t} + \sum_{d=1}^{M-1} (c_{d,t} - c_{d+1,t}) K(u_d - z_k) \right). \quad (18)$$

The dependence of the cost on the thresholds and on the neural networks parameters (z_k , depends on these parameters, as shown in (6)) has now been made explicit. The Parzen windows estimator makes (18) derivable ($K(z)$ is the integral of window $k(z)$). This estimate of the Bayes cost (10) is proposed as the loss function to be minimized during the training of the neural network classifier.

4.3. Training algorithm

A gradient descent training algorithm is used to minimize (18). The parameters $\mathbf{w}$ and the thresholds $\mathbf{u}$ are iteratively updated as²

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \left. \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}(n), \mathbf{u}=\mathbf{u}(n)}, \quad (19)$$

$$\mathbf{u}(n+1) = \mathbf{u}(n) - \mu \left. \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial \mathbf{u}} \right|_{\mathbf{w}=\mathbf{w}(n), \mathbf{u}=\mathbf{u}(n)}, \quad (20)$$

where μ is the step-size parameter. The gradient expression for each iteration can be computed sample-by-sample, batch (using the whole training set) or mini-batch (using a sub-set of the training set in each step). The sample-by-sample expression of the gradient for the network parameters $\mathbf{w}$ is

$$\frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial \mathbf{w}} = \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial z_k} \frac{\partial z_k}{\partial \mathbf{w}}. \quad (21)$$

The second term on the right-hand side of (21) is independent of the cost function: it only depends on the network architecture. The first term, for a pattern of class t , is

$$\left. \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial z_k} \right|_{k \in S_t} = \frac{\pi_t}{N_t} \sum_{d=1}^{M-1} (c_{d+1,t} - c_{d,t}) k(u_d - z_k). \quad (22)$$

The expression for the gradient of the thresholds is simpler, because it does not depend on the neural network architecture as the thresholds do not depend explicitly on the network output. To simultaneously update weights and thresholds, we will use the sample-by-sample contribution of a pattern $\mathbf{x}_k$ of class t with output z_k to the gradient

$$\left. \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial u_d} \right|_{k \in S_t} = \frac{\pi_t}{N_t} (c_{d,t} - c_{d+1,t}) k(u_d - z_k). \quad (23)$$

In many cases, the probabilities of the different classes are estimated from the training set, $\pi_t = \frac{N_t}{N}$. In these cases, by defining the normalized costs $\tilde{c}_{d,t} = \frac{c_{d,t}}{N}$ (a normalization of these costs does not affect the solution of the problem) more compact expressions can be obtained for (18), (22) and (23)

$$\hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u}) = \sum_{t=1}^M \left(\tilde{c}_{M,t} N_t + \sum_{k \in S_t} \sum_{d=1}^{M-1} (\tilde{c}_{d,t} - \tilde{c}_{d+1,t}) K(u_d - z_k) \right), \quad (24)$$

$$\left. \frac{\partial \hat{\mathcal{B}}_C(\mathbf{w}, \mathbf{u})}{\partial z_k} \right|_{k \in S_t} = \sum_{d=1}^{M-1} (\tilde{c}_{d+1,t} - \tilde{c}_{d,t}) k(u_d - z_k), \quad (25)$$

² For the sake of simplicity, the simplest gradient descent approach is shown. Obviously, some other stochastic optimization approaches can also be employed, such as including momentum, or Adam optimization, just to cite some examples.

$$\left. \frac{\partial \hat{\mathcal{B}}_c(\mathbf{w}, \mathbf{u})}{\partial u_d} \right|_{k \in S_t} = (\bar{c}_{d,t} - \bar{c}_{d+1,t})k(u_d - z_k). \quad (26)$$

With respect to the choice of the Parzen window, $k(z)$, it must be remarked that the goal is not to obtain good estimates of the conditional distributions $f_{Z|Y}(z|t)$ by (12), but to obtain a good classification performance³ (a discussion about this subject can be found in [23]).

4.4. Computational burden

To analyze the computational burden of the training algorithm, a multiclass neural network classifier with M neurons in the output layer (one per class), whose burden is well-known if the network architecture is given, will be used as a reference. The updating expressions of the proposed method are compared with those associated to a couple of well-known loss functions, such as the Mean Squared Error (MSE) and the Binary Cross-Entropy (BCE) losses, which are, respectively

$$\mathcal{L}_M(\mathbf{w}) = (y_k - z_k)^2, \quad (27)$$

$$\mathcal{L}_B(\mathbf{w}) = -y_k \log(z_k) - (1 - y_k) \log(1 - z_k). \quad (28)$$

For every loss function $\mathcal{L}(\mathbf{w})$ the contribution of a pattern $\mathbf{x}_k$ with output z_k to the gradient can be written as

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}(\mathbf{w})}{\partial z_k} \frac{\partial z_k}{\partial \mathbf{w}} \quad (29)$$

where the second term on the right-hand side does not depend on the loss but only on the network architecture, as in (21). For MSE and BCE

$$\frac{\partial \mathcal{L}_M(\mathbf{w})}{\partial z_k} = -2(y_k - z_k), \quad (30)$$

$$\frac{\partial \mathcal{L}_B(\mathbf{w})}{\partial z_k} = -\frac{y_k}{z_k} + \frac{1 - y_k}{1 - z_k}. \quad (31)$$

In a multiclass neural network, M terms (one per class) such as (30) or (31) must be computed for a pattern. With the proposed cost function, the computation of (22) or (25) for a pattern requires the addition of $M - 1$ terms and the evaluation of the window function. Taking into account that $\frac{\partial z_k}{\partial \mathbf{w}}$ is independent of the cost function, and that this term is the main responsible of the computational burden (specially in deep networks because it is back-propagated), it can be concluded that the computational complexity using the proposed cost function is of the same order of magnitude than that required for a multiclass neural network classifier using MSE or BCE loss functions.⁴ The update of thresholds $\mathbf{u}$ has not been considered in the discussion: The gradient expressions (23) or (26) do not depend on the network architecture, and the number of thresholds in a practical network is much lower than the number of network parameters in $\mathbf{w}$.

5. Experiments

In this section, after presenting appropriate figures of merit to measure the performance in imbalanced ordinal classification methods, experiments with a set of real data sets to evaluate the performance of the proposed methods will be presented.

³ In fact, these distributions are not estimated during the training procedure, but the formulation by using the Parzen window method allows to obtain closed form expressions for the gradient of the cost function (18), as in (22) and (23).

⁴ In a practical implementation, the small difference is mainly dependent on the cost for evaluating the Parzen windows $k(\cdot)$, which basically depends on the implementation platform.

5.1. Figures of merit

The choice of an appropriate figure of merit to compare the performance of different methods is very important in any application. In conventional classifications problems, the usual choice is the accuracy (or its complement, the probability of error), which for a labeled data set is defined as

$$Acc = \frac{1}{N} \sum_{k=1}^N I(\hat{y}_k = y_k), \quad (32)$$

where $I(\cdot)$ denotes the indicator function that returns 1 if the argument is true and 0 if it is false. Although accuracy is a reasonable performance measure for a balanced classification problem, in imbalanced problems it over-represents the accuracy in the majority classes and under-represents the accuracy in the minority classes. For this reason, it is frequently replaced by the average or balanced accuracy

$$AAcc = \frac{1}{M} \sum_{k=1}^M Acc_t, \quad (33)$$

where Acc_t is the accuracy in the detection of class t

$$Acc_t = \frac{1}{N_t} \sum_{k \in S_t} I(\hat{y}_k = y_k). \quad (34)$$

For ordinal problems, the figure of merit must include the distance between the true class and the decision. The Mean Absolute Error (MAE), which includes this distance, is a typical figure of merit for this kind of problems

$$MAE = \frac{1}{N} \sum_{k=1}^N |\hat{y}_k - y_k|. \quad (35)$$

In imbalanced ordinal problems, to avoid the over-representations of the performance for the majority classes, this measure is usually replaced by the average (or balanced) MAE

$$AMAE = \frac{1}{M} \sum_{t=1}^M MAE_t, \quad (36)$$

where MAE_t is the MAE for patterns of class t

$$MAE_t = \frac{1}{N_t} \sum_{k \in S_t} |\hat{y}_k - y_k|. \quad (37)$$

Cohen's kappa coefficient is a measure of agreement between observations that allows to discard agreements due to mere chance. It has been applied in the evaluation of classification algorithm and it has also been used as a loss function in ordinal problems [34]. The quadratic weighted Cohen's kappa coefficient is

$$\kappa^2 = 1 - \frac{\sum_{t=1}^M \sum_{d=1}^M (d - t)^2 o_{d,t}}{\sum_{t=1}^M \sum_{d=1}^M (d - t)^2 e_{d,t}}, \quad (38)$$

where $o_{d,t}$ is the observed disagreement for decision d and true class t and $e_{d,t}$ is the corresponding expected disagreement due to chance. This coefficient ranges from -1 (total disagreement) through 0 (random classification) to +1 (total agreement).

In this work, MAE and AMAE will be used to evaluate the performance in ordinal problems. Note that these two measures give a different importance to the misclassifications per class: MAE gives the same importance to every pattern, which in practice over-represents the performance for the majority classes because the contribution of a class is proportional to its probability (to be more precise, to the number of examples in the data set). And

Table 1
Experimental datasets and their main characteristics.

Dataset	Patterns	Attr.	Classes	Patterns per class
ERA	1000	4	9	(92,142,181,172, 158, 118, 88, 31, 18)
ESL	488	4	9	(2,12,38,100,116, 135,62,19,4)
LEV	1000	4	5	(93,280,403,197,27)
SWD	1000	10	4	(32,352,399,217)
automobile	205	71	6	(3,22,67,54,32,27)
balance-scale	625	4	3	(288,49,288)
bondrate	57	37	5	(6,33,12,5,1)
eucalyptus	736	91	5	(180,107,130,214,105)
newthyroid	215	5	3	(30,150,35)
pasture	36	25	3	(12,12,12)
squash-stored	52	51	3	(23,21,8)
squash-unstored	52	52	3	(24,24,4)
tae	151	54	3	(49,50,52)
toy	300	2	5	(35,87,79,68,31)
winequality-red	1599	11	6	(10,53,681,638,199,18)

AMAE gives the same importance to all classes, independently of their probability. Cohen's kappa coefficient will be used to assess the agreement between decisions and true classes that is obtained with the proposed method.

5.2. Datasets

Experiments have been performed with the 15 datasets that were used to evaluate several methods in [21], where each dataset was arranged in 30 partitions into training (around 3/4 of the data) and test (around 1/4 of the data) sets. The same partitions⁵ are considered here. The reported results average the performance for these 30 partitions. The basic characteristics of each dataset - number of patterns, number of attributes, number of classes and number of patterns per class - appear in Table 1.

5.3. Benchmark methods

The seven methods tested in [21] will be used as benchmark. These methods include two ordinal variants of class switching [37] to generate ensembles of classifiers that were proposed in [21], Arithmetic Ordinal Class Switching (AOCS), and Geometric Ordinal Class Switching (GOCS), along with the standard (Nominal) Class Switching (NCS) ensemble and a conventional ensemble that does not use class switching, denoted as Original ensemble (Orig). Moreover, results are provided for other three benchmark methods: reduction from ordinal regression to binary support vector machines (REDSVM) [9], the reformulation of Gaussian processes for ordinal regression (GPOR) [8], and the ORBoost method with all margins [38]. The details of the design of these methods can be found in [21].

5.4. Results

The proposed method has the flexibility provided by the parameters π_t and $c_{d,t}$ for $t, d \in \{1, 2, \dots, M\}$ to specify the relative importance of misclassifications as a function of the difference between the class label for the true class and the decision and at the same time the relative importance of each class in the objective function. To deal with the relative importance of misclassifications in an ordinal problem, we have considered

$$c_{d,t} = |d - t|. \quad (39)$$

⁵ Datasets and partitions are available at <http://uco.es/grupos/ayrna/orreview> (last access, October 2022)

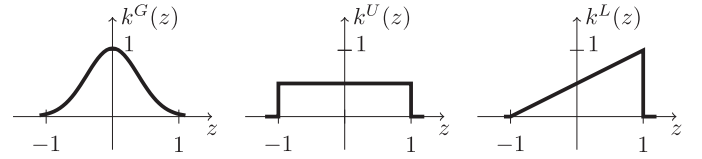


Fig. 2. Parzen windows used in the experiments, $k^X(z)$, $X \in \{G, U, L\}$, with G: Gaussian, U: uniform, L: linear.

To simulate two different situations that can happen in real problems, we have considered two different Bayesian Ordinal Neural Network (BONN) solutions:

- BONN(MAE): the importance of the class is proportional to its probability in the data set. For this kind of situation, the appropriate figure of merit to evaluate the performance of the different methods is MAE. The prior probability parameters of the method are

$$\pi_t = \frac{N_t}{N}. \quad (40)$$

- BONN(AMAE): the importance is the same for all classes, independently of their probability in the data set. In this scenario, which can happen in many imbalanced problems, the appropriate figure of merit is AMAE. The prior probability parameters for this case are

$$\pi_t = \frac{1}{M}. \quad (41)$$

Note that this setup is mathematically equivalent to consider the prior probability for each class as given by the data set but to modify the misclassification costs to be proportional to the inverse of the prior probability of the true class, i.e.

$$\pi_t = \frac{N_t}{N}, \quad c_{d,t} = \frac{1}{\pi_t} |d - t|. \quad (42)$$

In any case, the result is that all classes have the same weight in the proposed cost function (18) independently of their probability.

A multilayer perceptron with a single hidden layer with L neurons is used in the experiments. For each dataset, $L \in \{10, 20, 30, 40, 50, 100\}$ neurons, with both *tanh* and *ReLU* activation functions, have been tested. The neuron of the output layer has a linear activation function. Three different Parzen windows have been tested for each dataset: Gaussian, uniform and linear windows, which are plotted in Fig. 2. The uniform and linear windows are constrained to the domain $[-1, 1]$, and the variance of the Gaussian window is 0.1507 (99% of probability in $[-1, 1]$).

$$k^G(z) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{z^2}{2\sigma^2}}, \quad \text{with } \sigma^2 = 0.1507 \quad (43)$$

$$k^U(z) = \begin{cases} \frac{1}{2} & \text{if } |z| \leq 1 \\ 0 & \text{if } |z| > 1 \end{cases} \quad (44)$$

$$k^L(z) = \begin{cases} \frac{1}{2}(z+1) & \text{if } |z| \leq 1 \\ 0 & \text{if } |z| > 1 \end{cases} \quad (45)$$

The choice of the window weights differently the contribution of the samples to the updates during the training. The weight for a sample $\mathbf{x}_k$ depends on the distance between its network output, z_k , and the decision thresholds u_d (see the gradient Eqs. (25) and (26), which make the updates proportional to $k(u_d - z_k)$). The uniform window weights uniformly all samples at distance lower than 1. With the Gaussian window the weight decreases exponentially with the distance to the thresholds. The linear window weights

Table 2MAE (average $\pm$ standard deviation in the 30 partitions) for each data set.

Dataset	Best in [21]	BONN(MAE)
ERA	1.219 $\pm$ 0.044 (REDSVM)	1.193 $\pm$ 0.044
ESL	0.301 $\pm$ 0.035 (GPOR)	<u>0.291 $\pm$ 0.031</u>
LEV	0.410 $\pm$ 0.023 (REDSVM)	0.389 $\pm$ 0.026
SWD	0.440 $\pm$ 0.032 (GPOR)	<u>0.428 $\pm$ 0.029</u>
automobile	<u>0.263 $\pm$ 0.074</u> (NCS)	0.282 $\pm$ 0.063
balance-scale	0.001 $\pm$ 0.004 (REDSVM)	0.017 $\pm$ 0.008
bondrate	0.531 $\pm$ 0.110 (ORBoost)	0.438 $\pm$ 0.088
eucalyptus	0.331 $\pm$ 0.038 (GPOR)	0.352 $\pm$ 0.024
newthyroid	0.029 $\pm$ 0.022 (REDSVM)	0.017 $\pm$ 0.015
pasture	<u>0.219 $\pm$ 0.147</u> (NCS)	0.229 $\pm$ 0.075
squash-stored	0.327 $\pm$ 0.122 (AOCS/GOCS)	<u>0.281 $\pm$ 0.111</u>
squash-unstored	0.161 $\pm$ 0.085 (AOCS/GOCS)	0.096 $\pm$ 0.056
tae	0.461 $\pm$ 0.060 (REDSVM)	0.494 $\pm$ 0.073
toy	0.024 $\pm$ 0.013 (REDSVM)	0.049 $\pm$ 0.017
winequality-red	0.348 $\pm$ 0.019 (GOCS)	0.410 $\pm$ 0.014

Table 3AMAE (average $\pm$ standard deviation in the 30 partitions) for each data set.

Dataset	Best in [21]	BONN(AMAE)
ERA	1.370 $\pm$ 0.099 (AOCS)	1.277 $\pm$ 0.092
ESL	0.459 $\pm$ 0.116 (REDSVM)	<u>0.432 $\pm$ 0.070</u>
LEV	0.601 $\pm$ 0.051 (AOCS)	0.518 $\pm$ 0.056
SWD	0.576 $\pm$ 0.039 (ORBoost)	0.461 $\pm$ 0.046
automobile	<u>0.313 $\pm$ 0.120</u> (GOCS)	0.346 $\pm$ 0.104
balance-scale	0.001 $\pm$ 0.003 (REDSVM)	0.015 $\pm$ 0.013
bondrate	0.839 $\pm$ 0.260 (ORBoost)	<u>0.810 $\pm$ 0.243</u>
eucalyptus	0.362 $\pm$ 0.040 (GPOR)	0.381 $\pm$ 0.029
newthyroid	0.048 $\pm$ 0.040 (REDSVM)	0.026 $\pm$ 0.034
pasture	<u>0.219 $\pm$ 0.147</u> (NCS)	0.229 $\pm$ 0.075
squash-stored	0.368 $\pm$ 0.129 (ORBoost)	<u>0.337 $\pm$ 0.163</u>
squash-unstored	0.170 $\pm$ 0.137 (NCS)	<u>0.153 $\pm$ 0.084</u>
tae	0.459 $\pm$ 0.059 (REDSVM)	0.494 $\pm$ 0.082
toy	0.024 $\pm$ 0.016 (REDSVM)	0.054 $\pm$ 0.016
winequality-red	0.952 $\pm$ 0.076 (AOCS)	0.781 $\pm$ 0.121

linearly and asymmetrically the samples at distance lower than 1, with a higher weight for samples below the thresholds.

The best network size, activation function, and Parzen window for each data set and performance criterion have been selected by cross-validation (3-fold cross-validation with the training set).

Tables 2 and 3 show the resulting MAE and the AMAE, respectively. To simplify the comparison, in both tables only the result of the best benchmark method in [21] for each data set has been included (the detailed results of each benchmark method for each data set can be found in [21]).

The best (lowest) average value of MAE/AMAE for each data set is underlined, and boldface highlights the cases where the difference is significant. We define as significant those differences where the hypothesis that the two means are equivalent can be rejected with a level of significance equal to 5%, i.e., those datasets where the absolute value of the difference between mean values is higher than 1.65σ [39], σ being the estimated deviation of the difference between the MAE/AMAE of the two methods

$$\sigma = \sqrt{\frac{\sigma_{bench}^2}{n} + \frac{\sigma_{prop}^2}{n}}, \quad (46)$$

where σ_{bench} denotes the standard deviation for the best benchmark method in [21], σ_{prop} is the standard deviation of the proposed method, and $n = 30$ is the number of averaged partitions for both methods.

It can be seen that BONN(MAE) obtains the best MAE in 8 of the 15 data sets, with 5 significant wins, and with 5 significant losses in the remaining 7 data sets where the best method in [21] has the best MAE. The best results in AMAE are obtained by

Table 4Wins/Ties/Losses of the BONN(MAE) method in MAE against each one of the benchmark methods (considering wins with a level of significance of 5%), and T paired and Wilcoxon paired tests, including p -value and the hypothesis for the 5% of significance.

Method	Wins/Ties/Losses of BONN(MAE)	T-Test p -value (H)	Wilcoxon p -value (H)
Orig	13/2/0	0.0003 (1)	0.0001 (1)
NCS	11/3/1	0.0127 (1)	0.0128 (1)
AOCS	9/5/1	0.0178 (1)	0.0067 (1)
GOCS	9/5/1	0.0183 (1)	0.0063 (1)
GPOR	11/3/1	0.0052 (1)	0.0014 (1)
ORBoost	12/2/1	0.0030 (1)	0.0009 (1)
REDSVM	11/1/3	0.0164 (1)	0.0215 (1)

Table 5Wins/Ties/Losses of the BONN(AMAE) method in AMAE against each one of the benchmark methods (considering wins with a level of significance of 5%), and T paired and Wilcoxon paired tests, including p -value and the hypothesis for the 5% of significance.

Method	Wins/Ties/Losses of BONN(AMAE)	T-Test p -value (H)	Wilcoxon p -value (H)
Orig	12/3/0	0.0018 (1)	0.0003 (1)
NCS	10/5/1	0.0087 (1)	0.0016 (1)
AOCS	9/6/0	0.0106 (1)	0.0012 (1)
GOCS	9/6/0	0.0101 (1)	0.0012 (1)
GPOR	13/0/2	0.0007 (1)	0.0002 (1)
ORBoost	11/4/0	0.0003 (1)	0.0001 (1)
REDSVM	10/2/3	0.0036 (1)	0.0026 (1)

BONN(AMAE) in 9 of the 15 data sets, with 5 significant wins, and with 4 significant losses in the other 6 data sets.

It is important to remark that the best result in [21] for each data set is obtained by different methods. These methods are indicated in Tables 2 and 3. So, the performance of the proposed procedures must be considered very good. To further support this conclusion, Tables 4 and 5 presents the number of wins, ties and losses of BONN(MAE) in MAE and BONN(AMAE) in AMAE, respectively, against each benchmark method (wins and losses are defined with the significance level of 5%, and ties are the cases where the hypothesis of the means being equivalent cannot be rejected with this significance level). These tables contain also the p -value and the hypothesis for a paired T test and a paired Wilcoxon test. It can be seen that BONN(MAE) has a minimum of 9 wins in MAE – against AOCS and GOCS with only 1 loss – and a maximum of 3 losses, against REDSVM but with 11 wins against this method. In AMAE, BONN(AMAE) has also a minimum of 9 wins – against AOCS and GOCS, now with no losses – and a maximum of 3 losses against REDSVM, but with 10 wins against it. In all cases, the null hypothesis assuming that the true mean difference is zero is discarded with both the T test and the Wilcoxon test. These results permit to conclude that the proposed approach has a remarkably high performance: In a pair comparison it provides the best average results and the best wins/losses figure against every benchmark method. The difference in performance is statistically significant in all cases.

To test the differences between more than two models, the Friedman test [40] is a well known and very used method. Figure 3 plots the means and ranks obtained in the Friedman test, both for MAE and AMAE. This test also shows a clear advantage in the performance of the proposed method with respect to all the benchmark methods, specially in AMAE, the scenario where imbalance is taken into account.

One of the main characteristics of the proposed method is the flexibility that is provided by the Bayesian formulation. This feature allows it to provide good results in different scenarios with different requirements. Although the method is designed to improve

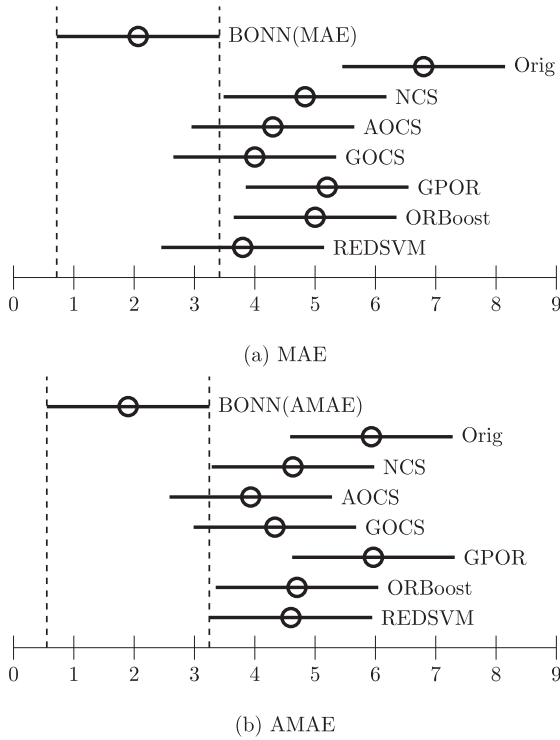


Fig. 3. Average means and ranks for Friedman test applied to the 8 methods under comparison for both MAE and AMAE (better models: on the left).

Table 6
Average figures of merit in the 15 datasets.

Method	Acc	AAcc	MAE	AMAE	Cohen's κ^2
MLP (Softmax)	0.6842	0.6132	0.3938	0.5089	0.7180
BONN(Acc)	0.6772	0.5825	0.3990	0.5603	0.6701
BONN(AAcc)	0.6274	<u>0.6414</u>	0.4813	0.4836	0.6769
BONN(MAE)	0.6841	0.5877	0.3645	0.5137	0.7085
BONN(AMAE)	0.6286	0.6441	0.4415	0.4270	<u>0.7177</u>

the metrics that are related with ordinal problems, including imbalance, the method can also be used to match different requirements. Just as an example, it can be used to maximize accuracy or balanced accuracy. If the decision costs are now

$$c_{d,t} = \begin{cases} 0, & d = t \\ 1, & d \neq t \end{cases} \quad (47)$$

these costs along with (40) and (41) can be used to maximize (32) and (33), respectively: BONN(Acc) and BONN(AAcc) are used to name these two new approaches. Table 6 compares the average performance obtained in the 15 datasets using the proposed model with the 4 different configurations. The figures of merit are accuracy, balanced accuracy, MAE and AMAE (for the sake of a more compact presentation, the standard deviations are not included in the table, but have been used to determine the wins/ties in each figure of merit as in the previous presented results). Here, the best results for each figure of merit are highlighted in boldface, while those without a significant difference with the best result (ties, using (46) as before) are underlined. A conventional multi-class MLP with softmax activation in the output layer has also been included in the comparison as a benchmark for accuracy. To isolate the effect of the cost function in the results, the same basic architecture has been used with all methods in all datasets: an MLP with 20 neurons in the hidden layer and *tanh* activation function (note that the output layer is different for the MLP with softmax activation).

With respect to imbalance, Table 6 shows the expected results: There is a trade-off between the metrics that consider and that do not consider the imbalance to measure the performance. The methods designed to improve Acc or MAE have worse values for AAcc or AMAE (and vice versa). The results in Table 6 also show that to use the ordinal information in the classification task can be useful. In accuracy, the MLP with softmax provides slightly better results than BONN(Acc), as expected (in general, multi-class approaches with softmax have shown in the literature better results than regression based approaches in terms of accuracy for classification problems). But the accuracy of BONN(MAE), which is designed taking the order into account in the classification costs $c_{d,t}$ to minimize MAE, obtains an equivalent accuracy at the same time that obtains the best (lowest) MAE. A similar behavior is observed when the imbalance is considered: For balanced accuracy, an equivalent performance is obtained with BONN(AMAE), designed taking into account the order, and BONN(AAcc), which does not include the order in its design but is designed to maximize balanced accuracy. At the same time, BONN(MAE) obtains the best AMAE, as expected. Therefore, to improve the ordinal classification (MAE/AMAE) can help to improve the accuracy (Acc/AAcc). We want to remark that Table 6 shows the average results in the 15 datasets. In some datasets when MAE/AMAE is improved Acc/AAcc increases. But in other datasets a trade-off between MAE and Acc (or AMAE and AAcc) has been observed: To increase one of them tends to decrease the other one (when close to the best metrics, obviously). Finally, Cohen's kappa coefficient shows that all methods have a good agreement between decisions and true classes, far away from agreement by chance, with MLP with softmax and BONN(AMAE) having the best values for κ^2 .

The previous experiments show that the proposed method allows to simultaneously deal with ordinal problems and with the imbalance, as BONN(AMAE) does with the parameters in (42). In this case all classes have the same weight in the loss function independently of their probabilities. But the Bayesian formulation provides also the flexibility to consider the relative importance among classes. This can be useful in applications where the detection of a class, which can be the minority class, is highly relevant to end users. As an example of this flexibility, Table 7 compares the confusion matrices obtained in the LEV dataset with the MLP (Softmax), BONN(MAE), BONN(AMAE) and BONN(Emph-5), which denotes a solution emphasizing class 5, the minority class in LEV (only 2.7% of samples, compared to 40.3% for class 3, the majority class). This solution is using the following parameters

$$\pi_t = \frac{N_t}{N}, \quad c_{d,t} = \begin{cases} \frac{1}{\pi_t} |d - t|, & t \neq 5 \\ \frac{2}{\pi_t} |d - t|, & t = 5 \end{cases} \quad (48)$$

To emphasize the importance of class 5, its contribution to the loss is now doubled. The diagonal of the matrices, which contains the per class accuracy, Acc_t in (34), has been highlighted (boldface) to facilitate the analysis. Additionally, Acc, AAcc (it is equal to the average of the values in the diagonal of the confusion matrix), MAE and AMAE are provided for every solution. The MLP and BONN(MAE) show a similar behavior: Both approaches have a better accuracy for the majority classes, and the minority class (class 5) is almost ignored. The conventional MLP obtains a slightly better accuracy, and BONN(MAE) obtains a slightly better MAE by reducing in general the misclassification errors with higher class differences, as expected. Otherwise, BONN(AMAE) increases notably AMAE by forcing a more uniform per class accuracy (note that also average accuracy, AAcc, is notably improved). Finally, BONN(Emph-5) is able to improve the accuracy of class 5, even although it is the minority class, obviously at the price of a lower accuracy for the other classes (specially for the closest class) and a slightly worse AMAE than BONN(AMAE).

Table 7

Confusion matrices and several performance metrics obtained in the LEV dataset.

Acc=0.630, AAcc=0.488, MAE=0.405, AMAE=0.609					
0.4720	0.3774	0.1261	0.0245	0.0000	
0.0507	0.6686	0.2574	0.0228	0.0005	
0.0000	0.1835	0.6905	0.1252	0.0008	
0.0000	0.0234	0.3651	0.6058	0.0057	
0.0000	0.0796	0.1020	0.8142	0.0041	
(a) MLP (Softmax)					
Acc=0.628, AAcc=0.469, MAE=0.402, AMAE=0.644					
0.4992	0.3318	0.1578	0.0112	0.0000	
0.0467	0.6542	0.2907	0.0085	0.0000	
0.0001	0.1559	0.7953	0.0485	0.0001	
0.0000	0.0161	0.5906	0.3915	0.0019	
0.0000	0.0717	0.2192	0.7019	0.0071	
(b) BONN(MAE)					
Acc=0.534, AAcc=0.584, MAE=0.530, AMAE=0.516					
0.6945	0.1436	0.1155	0.0389	0.0075	
0.1900	0.5777	0.1694	0.0560	0.0069	
0.0144	0.2041	0.4210	0.3408	0.0198	
0.0000	0.0361	0.1094	0.6161	0.2383	
0.0000	0.0514	0.0427	0.2973	0.6086	
(c) BONN(AMAE)					
Acc=0.477, AAcc=0.567, MAE=0.624, AMAE=0.549					
0.6640	0.1666	0.1055	0.0507	0.0132	
0.1797	0.5525	0.1818	0.0696	0.0164	
0.0122	0.1977	0.3825	0.3205	0.0871	
0.0000	0.0371	0.0924	0.4362	0.4343	
0.0000	0.0154	0.0670	0.1157	0.8019	
(d) BONN(Emph-5)					

6. Conclusions

The proposed method combines the flexibility of the Bayesian formulation with the advantages of the threshold-based regression methods, where the decision is based on thresholds over a latent variable (the output of a neural network in this case). The Bayesian formulation allows to deal with imbalance and with the relative relevance of the different classes, independently of their distribution. This makes the proposed method flexible to work in very different scenarios and robust, as shown in the experiments. The regression of the input patterns in a single output helps to force the ordinal structure of the problem in the learning machine. Moreover, as the values of the thresholds are learned, there is not a prior assumption about the meaningful numeric difference between classes, but this difference is learned through the thresholds.

Another advantage of the proposed method is that the loss function (18) can be used with every neural architecture. In this manuscript, an MLP with a single hidden layer has been employed in the experiments. But it can be used with deep MLPs or with other deep networks, such as CNNs, and allows to use techniques such as drop-out or batch-normalization, just to mention some examples, to exploit all the potential of deep solutions. This cost function does not increase the computational burden of the training process.

The proposed solution inherits the main limitations of neural networks, in particular those related with their non-deterministic nature, and cost-sensitive methods. The computational burden is relatively high, compared with other solutions, and hyperparameters have to be obtained, typically by cross validation. In this case a new hyperparameter is included: the Parzen window. However, this hyperparameter can be easily selected by cross-validation and some preliminary results show that the sensitivity of the results to the choice of this hyperparameter decreases in deep networks (the

same behavior has been observed when the Parzen windows are used to train neural networks for binary example dependent cost (EDC) problems [24]). Cost-sensitive methods assume that the cost matrix is available, as it happens if the goal is to minimize MAE or AMAE, and the matrix can also be used to consider the relative importance between classes. However, in some cases the relationship of this matrix with the requirements of the application may be unclear.

Moving forward from this work, to explore the most appropriate architectures to deal with ordinal problems is a research direction which can produce valuable results. How to introduce diversity mechanisms for obtaining ensembles that offer improved performance is also interesting.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data is public (a link has been provided).

Acknowledgment

This work was partially supported by Spanish Ministry of Science and Innovation through Thematic Network “MAPAS”(TIN2017-90567-REDT) and by BBVA Foundation through “2-BARBAS” research grant. Funding for APC: Universidad Carlos III de Madrid (Read & Publish Agreement CRUE-CSIC 2023).

References

- [1] P.A. Gutiérrez, M. Pérez-Ortiz, J. Sánchez-Monedero, F. Fernández-Navarro, C. Hervás-Martínez, Ordinal regression methods: survey and experimental study, *IEEE Trans. Knowl. Data Eng.* 28 (2016) 127–146.
- [2] P. McCullagh, Regression models for ordinal data, *J. R. Stat. Soc. Ser. B (Methodological)* 42 (1980) 109–142.
- [3] L. Gaudette, N. Japkowicz, Evaluation methods for ordinal classification, in: Y. Gao, N. Japkowicz (Eds.), *Advances in Artificial Intelligence. Canadian AI 2009*, Springer LNCS, Vol. 5549, 2009, pp. 207–210.
- [4] J.S. Cardoso, R. Sousa, Measuring the performance of ordinal classification, *Int. J. Pattern Recognit.* 25 (2011) 1173–1195.
- [5] J.P. Amorim, I. Domingues, P.H. Abreu, J. Santos, Interpreting deep learning models for ordinal problems, in: *Proc. of the European Symp. on Artificial Neural Networks (ESANN)*, 2018, pp. 373–378. Bruges, Belgium
- [6] L. Kook, L. Herzog, T. Hothorn, O. Dürr, B. Sick, Deep and interpretable regression models for ordinal outcomes, *Pattern Recognit.* 122 (2022) 1–11.
- [7] S. Kramer, G. Widmer, B. Pfahringer, M. De Groove, Prediction of ordinal classes using regression trees, *Fundam. Inform.* 47 (2001) 1–13.
- [8] W. Chu, Z. Ghahramani, Gaussian processes for ordinal regression, *J. Mach. Learn. Res.* 6 (2005) 1019–1041.
- [9] H.T. Lin, L. Li, Reduction from cost-sensitive ordinal ranking to weighted binary classification, *Neural Comput.* 24 (2012) 1329–1367.
- [10] P.A. Gutiérrez, P. Tiño, C. Hervás-Martínez, Ordinal regression neural networks based on concentric hyperspheres, *Neural Netw.* 59 (2014) 51–60.
- [11] F. Tang, P. Tiño, Ordinal regression based on learning vector quantization, *Neural Netw.* 93 (2017) 76–88.
- [12] W. Cao, V. Mirjalili, S. Raschka, Rank consistent ordinal regression for neural networks with applications to age estimation, *Pattern Recognit. Lett.* 140 (2020) 325–331.
- [13] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue, G. Bing, Learning from class-imbalanced data: review of methods and applications, *Expert Syst. Appl.* 73 (2017) 220–239.
- [14] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, *J. Artif. Intell. Res.* 16 (2002) 321–357.
- [15] S. Ertekin, J. Huang, L. Bottou, C.L. Giles, Learning on the border: active learning for imbalanced data classification, in: *Proc. of the 16th ACM Conf. on Information and Knowledge Management*, 2007, pp. 127–136. Lisbon, Portugal
- [16] M. Galar, A. Fernández, E. Barrenechea, H. Bustince, F. Herrera, A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches, *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.* 42 (2012) 463–484.
- [17] Z. Zhu, Z. Wang, D. Li, Y. Zhu, W. Du, Geometric structural ensemble learning for imbalanced problems, *IEEE Trans. Cybern.* 50 (2020) 1617–1629.

- [18] A. Fernández, V. López, M. Galar, M.J. del Jesus, F. Herrera, Analysing the classification of imbalanced data-sets with multiple classes: binarization techniques and ad-hoc approaches, *Knowl. Based Syst.* 42 (2013) 97–110.
- [19] M. Pérez-Ortiz, P.A. Gutiérrez, C. Hervás-Martínez, X. Yao, Graph-based approaches for over-sampling in the context of ordinal regression, *IEEE Trans. Knowl. Data Eng.* 27 (2015) 1233–1245.
- [20] I. Domingues, J.P. Amorim, P.H. Abreu, H. Duarte, J. Santos, Evaluation of over-sampling data balancing techniques in the context of ordinal classification, in: 2018 Intl. Joint Conf. on Neural Networks (IJCNN), 2018, pp. 1–8. Rio de Janeiro (Brasil)
- [21] P. Gutiérrez, M. Pérez-Ortiz, A. Suárez, Class switching ensembles for ordinal regression, in: *Proc. Intl. Work Conf. on Artificial Neural Networks*, Vol. 10350, Springer LNCS, Cádiz, Spain, 2017, pp. 408–419.
- [22] M. Kukar, I. Kononenko, Cost-sensitive learning with neural networks, in: *Proc. European Conf. Artificial Intelligence*, 1998, pp. 445–449.
- [23] M. Lázaro, M.H. Hayes, A.R. Figueiras-Vidal, Training neural network classifiers through Bayes risk minimization applying unidimensional Parzen windows, *Pattern Recognit.* 77 (2018) 204–215.
- [24] M. Lázaro, A.R. Figueiras-Vidal, A Bayes-risk minimization machine for example-dependent cost classification, *IEEE Trans. Cybern.* 51 (2021) 3524–3534.
- [25] M. Desprez, K. Zawada, D. Ramp, Overcoming the ordinal imbalanced data problem by combining data processing and stacked generalization, *Mach. Learn. Appl.* 7 (2022).
- [26] S.R. Kotsiantis, P.E. Pintelas, A cost sensitive technique for ordinal classification problems, in: *Methods and Applications of Artificial Intelligence*. Springer LNCS, Vol. 3025, 2004, pp. 220–229.
- [27] J. Cid-Sueiro, J.I. Arribas, S. Urbán-Muñoz, A.R. Figueiras-Vidal, Cost functions to estimate a posteriori probabilities in multiclass problems, *IEEE Trans. Neural Netw.* 10 (1999) 645–656.
- [28] A.C. Bahnsen, A. Stojanovic, D. Aouada, B. Ottersten, Improving credit card fraud detection with calibrated probabilities, in: *Proc. of the 14th Intl. Conf. on Data Mining*, SIAM, Philadelphia, USA, 2014, pp. 677–685.
- [29] D.F. Specht, Probabilistic neural networks, *Neural Netw.* 3 (1990) 109–118.
- [30] G.A. Babich, O.I. Camps, Weighted Parzen windows for pattern classification, *IEEE Trans. Pattern Anal. Mach. Intell.* 18 (1996) 567–570.
- [31] C. Beckham, C. Pal, Unimodal probability distributions for deep ordinal classification, in: *Proc. of Machine Learning Research*, Vol. 70, 2017, pp. 411–419.
- [32] T. Albuquerque, R. Cruz, J.S. Cardoso, Quasi-unimodal distributions for ordinal classification, *Mathematics* 10 (2022) 1–13.
- [33] A. Rios, R. Kavuluru, Ordinal convolutional neural networks for predicting RDoC positive valence psychiatric symptom severity scores, *J. Biomed. Inform.* 75 (2017) S85–S93.
- [34] J. de la Torre, D. Puig, A. Valls, Weighted kappa loss function for multi-class classification or ordinal data in deep learning, *Pattern Recognit. Lett.* 105 (2018) 144–154.
- [35] V.M. Vargas, P.A. Gutiérrez, C. Hervás-Martínez, Cumulative link models for deep ordinal classification, *Neurocomputing* 401 (2020) 48–58.
- [36] E. Parzen, On the estimation of a probability density function and the mode, *Ann. Math. Stat.* 33 (1962) 1065–1076.
- [37] L. Breiman, Randomizing outputs to increase prediction accuracy, *Mach. Learn.* 40 (2000) 229–242.
- [38] H.T. Lin, L. Li, Large-margin thresholded ensembles for ordinal regression: Theory and practice, in: *Proc. of the 17th Intl. Conf. on Algorithmic Learning Theory*, Vol. 4262, Springer LNCS, Barcelona, Spain, 2006, pp. 319–333.
- [39] V.E. Cangelosi, *Basic Statistics: A Real World Approach*, 2nd ed., West Publishing Co, Rochester, New York, 1980.
- [40] J. Demsar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30.

Marcelino Lázaro was born in Carriazo, Cantabria, Spain, in 1972. He received the Ingeniero de Telecomunicación and Doctor Ingeniero de Telecomunicación degrees from the Universidad de Cantabria, Spain, in 1996 and 2001, respectively. From 1996 to 2002, he was with the Departamento de Ingeniería de Comunicaciones, Universidad de Cantabria. In 2003, he joined the Departamento de Teoría de la Señal y Comunicaciones, Universidad Carlos III de Madrid, Spain. His research interest includes digital signal processing, detection, estimation, and statistical learning methods.

Aníbal R. Figueiras-Vidal received the Telecommunication Engineer (Hons.) degree from Universidad Politécnica de Madrid, Madrid, Spain, in 1973, and the Doctor (Hons.) degree from Universidad Politécnica de Barcelona, Barcelona, Spain, in 1976. He also received Honoris Causa Doctor degrees from Universidad de Vigo, Vigo, Spain, in 1999, and Universidad San Pablo, Arequipa, Peru, in 2011. He is a Professor of signal theory and communications with Universidad Carlos III de Madrid, Leganés (Madrid), Spain. He has authored or co-authored more than 300 journal and conference papers, and he has been the principal researcher in almost 100 research projects and contracts. His current research interests are digital signal processing and machine learning, including their applications. Dr. Figueiras-Vidal has been Chair of IEEE Spain Section and IEEE Spain Joint Chapter on Signal Processing and Communications. Dr. Figueiras-Vidal is a member of the Royal Academy of Engineering of Spain. He was its President from 2007 to 2011. He is also Corresponding Member of the Mexico Academy of Engineering.